

SubmitEase

Enrollment No. (s) : 23103298, 23103299, 23103374

Name of Student (s) : Aaditya Pratap Singh, Arpit Varshney, Vasu Tayal

Name of Supervisor : Dr. Ankit Vidyarthi



May 2026

Submitted in the Partial Fulfilment of the Degree of

Bachelor of Technology

In

Computer Science Engineering

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING &

INFORMATION TECHNOLOGY

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY , NOIDA

Students' Self Declaration for Open-Source libraries and other source code usage in Minor Project

We **Aaditya Pratap Singh (23103298)**, **Arpit Varshney (23103299)**, **Vasu Tayal (23103374)** hereby declare the following usage of the open-source code and prebuilt libraries in our minor project in V Semester with the consent of our supervisor. We also measure the similarity percentage of pre written source code and our source code and the same is mentioned below.

This measurement is true with best of our knowledge and abilities.

1. List of Pre-Built Libraries:

The following open-source libraries were used in the development of our project. Each library is licensed under permissive open-source licenses.

- React
- Node.js
- Prisma Postgres
- Vite
- Multer
- CORS
- Bcryptjs
- JSZip

2. List of Pre-Built Features in Libraries or Source Code:

We have used the following pre-built features from the above libraries with minimal modifications:

- Database Object Relational Mapping (Prisma)
- Routing Middleware (Express.js)
- Virtual DOM Rendering (React)
- Password Hashing (Bcryptjs)
- File Stream Handling (Multer)

3. Percentage of pre-written source code and source written by us

Percentage of pre-written source code: 30%

Percentage of source code written by us: 70%

Student ID	Student Name	Student Signature
23103298	Aaditya Pratap Singh	
23103299	Arpit Varshney	
23103374	Vasu Tayal	

Declaration by Supervisor

I, Dr. Ankit Vidyarthi declare that the above submitted project titled SubmitEase was conducted in my supervision. The project is original and neither the project was copied from external sources nor it was submitted earlier in IIIT. I authenticate this project.

Signature (Supervisor)

CERTIFICATE

This is to certify that the work titled **SubmitEase** submitted by **Aaditya Pratap Singh, Arpit Varshney, Vasu Tayal** in partial fulfilment for the **B.Tech** Computer Science And Engineering of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of this or any other degree or diploma.

Signature Of Supervisor	
Name Of Supervisor	Dr. Ankit Vidyarthi
Date	7 th May 2026
Signature Of HOD	

DECLARATION OF ORIGINALITY

We hereby declare that the minor project report entitled "SubmitEase", submitted to Jaypee Institute of Information Technology, Noida, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is an authentic and original record of our collaborative work carried out under the supervision and guidance of Dr. Ankit Vidyarthi

We further declare that the work reported in this project has not been submitted, either in part or in full, to any other university or educational institution for the award of any degree, diploma, or fellowship.

To the best of our knowledge, all sources of information, literature, and data used in this report have been duly acknowledged and properly cited in the references section.

Date: 7th May 2026

Place: Jaypee Institute Of Information Technology, Noida

Signatures of the Students:

(Signature) _____

Name: Aaditya Pratap Singh

Enrollment No.: 23103298

(Signature) _____

Name: Arpit Varshney

Enrollment No.: 23103299

(Signature) _____

Name: Vasu Tayal

Enrollment No.: 23103374

ACKNOWLEDGEMENT

We would like to express our sincere thanks and gratitude to our supervisor Dr. Ankit Vidyarthi for his willingness to give us valuable advice and direction whenever we approached him with a problem.

Your valuable guidance and support helped us in various phases of completion of this project. We will always be thankful to you in this regard.

Date	7 th May 2026		
Enrolment Numbers	23103298	23103299	23103374
Names of Students	Aaditya Pratap Singh	Arpit Varshney	Vasu Tayal
Signature Of Students			

ABSTRACT

The traditional management of academic conferences and journals often relies on fragmented communication channels, manual tracking, and disjointed workflows. This lack of a centralized platform creates significant inefficiencies for authors submitting manuscripts, peer reviewers evaluating content, and editors managing the publication lifecycle. There is a critical need for a secure, unified system that can streamline the submission and peer-review process while maintaining strict data integrity and managing distinct user permissions.

To address these administrative challenges, "SubmitEase" was developed as a modern, centralized web application. The platform is engineered using a robust full-stack architecture based on the PERN stack (PostgreSQL, Express, React, Node.js). Database management and schema modelling are handled efficiently using Prisma ORM and Supabase, guided by detailed Entity-Relationship (ER) diagrams. A core component of the methodology was the implementation of strict Role-Based Access Control (RBAC) to securely manage permissions across different user types in the journal workflow. Furthermore, the backend infrastructure is structured for scalable deployment and reliable API communication.

The development of SubmitEase resulted in a highly functional prototype capable of managing the end-to-end academic publication workflow. The platform successfully automates manuscript submissions, streamlines the assignment and tracking of peer reviews, and securely isolates data based on assigned user roles. The integration of Prisma with a structured PostgreSQL database ensures high-performance querying and data consistency. Overall, the system provides a seamless, intuitive interface that drastically reduces the manual overhead required to process academic papers.

By alleviating the administrative bottlenecks associated with academic publishing, SubmitEase provides significant value to educational institutions, conference organizers, and the broader research community. A more efficient peer-review and publication workflow accelerates the dissemination of scientific knowledge and discoveries. By allowing researchers and editors to focus their time on academic innovation rather than administrative overhead, SubmitEase fosters a more accessible, transparent, and collaborative global academic ecosystem.

TABLE OF CONTENTS

CHAPTER-1 INTRODUCTION.....	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Social & Environmental Relevance	3
1.4 Timeline	3
1.5 Scope.....	4
CHAPTER-2 LITERATURE SURVEY.....	5
2.1 Overview of Existing Work	5
2.1.1 Conference Management Systems	5
2.1.2 Academic Journal Management Systems	6
2.2 Summary Table	7
2.3 Limitations of Existing Methods.....	8
CHAPTER-3 SYSTEM DESIGN & METHODOLOGY	9
3.1 Proposed Methodology	9
3.1.1 System Architecture	10
3.1.2 System Features	14
3.2 Tool and Technique Selection.....	16
3.2.1 Frontend Technologies.....	16
3.2.2 Backend Technologies	17
3.2.3 Database and Storage	17
3.2.4 Build Tools and Version Control	18
CHAPTER-4 IMPLEMENTATION AND TESTING	19
4.1 Experimental Setup	19
4.2 Test Cases / Implementation Outputs	20
4.2.1 Testing Implementation	20
4.2.2 Backend Implementation	22
4.2.3 Frontend Implementation.....	24
CHAPTER-5 RESULTS & DISCUSSION.....	31
5.1 Result analysis.....	31
5.2 Performance Evaluation and Comparison with Existing Methods	32
5.2.1 Performance Evaluation.....	32
5.2.2 Comparison with Existing Methods.....	33

CHAPTER-6 CONCLUSION AND FUTURE SCOPE	35
6.1 Summary of Work.....	35
6.2 Impact on society, Environmental sustainability Ethical issues and compliance	35
6.3 Limitations of the Work.....	36
6.4 Future Scope	36
REFERENCES.....	37

LIST OF FIGURES

Figure Number	Title
Figure – 1.1	Timeline
Figure – 3.1	Data Flow Diagram
Figure – 3.2	Use Case Diagram
Figure – 3.3	Sequence Diagram
Figure – 3.4	Database Design (ER-Diagram)
Figure – 4.1	Frontend And Backend Testing
Figure – 4.2	Database Schema
Figure – 4.3	Home Page
Figure – 4.4	User Dashboard
Figure – 4.5	Author Portal
Figure – 4.6	Reviewer Portal
Figure – 4.7	Editor Portal
Figure – 4.8	Journal Host Portal
Figure – 4.9	EIC Portal
Figure – 4.10	Admin Portal

LIST OF TABLES

Table Number	Title
Table – 2.1	Summary Table
Table – 4.1	API Documentation
Table – 5.1	Comparison Table

CHAPTER-1

INTRODUCTION

1.1 Background

The dissemination of scientific knowledge, technological advancements, and academic research relies fundamentally on the publication of peer-reviewed journals and conference proceedings. Peer review acts as the cornerstone of academic integrity, ensuring that published research meets rigorous standards of quality, validity, and originality. However, over the past two decades, the global academic community has witnessed an exponential proliferation in research output. Driven by the expansion of higher education and the competitive nature of academic publishing, the sheer volume of manuscript submissions has created unprecedented administrative challenges for editorial boards, conference organizers, and academic institutions.

Historically, the management of these submissions was a highly manual, paper-based process reliant on postal systems. While the advent of the internet transitioned this workflow to digital formats, many institutions initially adopted fragmented solutions, utilizing basic email threads, local spreadsheets, and disjointed file-sharing services to manage submissions. This ad-hoc approach is highly prone to human error, resulting in lost manuscripts, delayed communications, and a lack of transparency for the authors awaiting decisions. As the scale of submissions grew, it became evident that generic communication tools were insufficient to handle the complex, multi-stage lifecycle of academic publishing—which involves receiving manuscripts, conducting initial editorial screenings, assigning appropriate peer reviewers, managing blind or double-blind review processes, collating structured feedback, and executing final editorial decisions.

Furthermore, the academic publication process requires strict data confidentiality and conflict-of-interest management. Unpublished research contains sensitive intellectual property that must be protected. Therefore, any system managing this workflow must possess granular security protocols, specifically Role-Based Access Control (RBAC), to ensure that users—whether they are submitting authors, evaluating reviewers, or managing editors—only have access to the data and functionalities explicitly permitted by their assigned roles.

In parallel with these administrative challenges, there has been a massive paradigm shift in software engineering. The rise of cloud computing and modern full-stack web development frameworks has made it possible to build highly scalable, responsive, and secure web applications. Technologies such as React.js for dynamic user interfaces, Node.js and Express.js for robust backend API routing, and PostgreSQL (managed via modern ORMs like Prisma and cloud platforms like Supabase) for relational data integrity, provide the perfect technological ecosystem to solve complex workflow problems.

SubmitEase is conceptualized against this operational and technological background. It leverages modern web development practices (the PERN stack) to replace fragmented legacy workflows with a unified, digital platform. By streamlining the end-to-end publication lifecycle, SubmitEase aims to alleviate the administrative bottlenecks of peer review, allowing researchers and editors to focus on advancing scientific discovery rather than managing paperwork

1.2 Problem Statement

Despite the availability of digital tools, many academic committees still rely on disjointed software—using combinations of basic web forms, email threads, and local spreadsheets—to manage manuscript submissions. This fragmented approach leads to several critical issues:

- **Inefficient Tracking:** Manuscripts are frequently delayed or lost in email threads, prolonging the peer-review cycle.
- **Lack of Transparency:** Authors lack real-time visibility into the status of their submissions.
- **Access Control Vulnerabilities:** Managing distinct permissions for Authors, Reviewers, and Editors is difficult without a unified system, increasing the risk of data breaches or conflicts of interest.
- **Administrative Overhead:** Editors spend excessive time manually routing papers rather than focusing on the academic quality of the journal.

Problem Statement: To design and develop SubmitEase, a centralized, scalable web application using the PERN stack (PostgreSQL, Express, React, Node.js) that automates the academic publication workflow, ensures data integrity through robust relational database modelling, and enforces strict Role-Based Access Control (RBAC) across diverse user hierarchies.

1.3 Social & Environmental Relevance

The development of SubmitEase carries notable social and environmental benefits:

Social Relevance: By reducing the administrative friction in academic publishing, SubmitEase accelerates the peer-review process. This faster turnaround allows critical scientific discoveries and technological advancements to reach the public and the broader research community more quickly. Furthermore, an organized, transparent system reduces bias and human error, fostering a more equitable academic ecosystem.

Environmental Relevance: SubmitEase promotes a completely paperless workflow. By digitizing manuscript submissions, reviewer annotations, and editorial decisions, the system eliminates the need for physical printing and physical archiving. Additionally, by utilizing optimized cloud database solutions (such as Supabase), the application reduces the reliance on localized, energy-consuming physical servers, contributing to a greener, more sustainable IT infrastructure.

1.4 Timeline

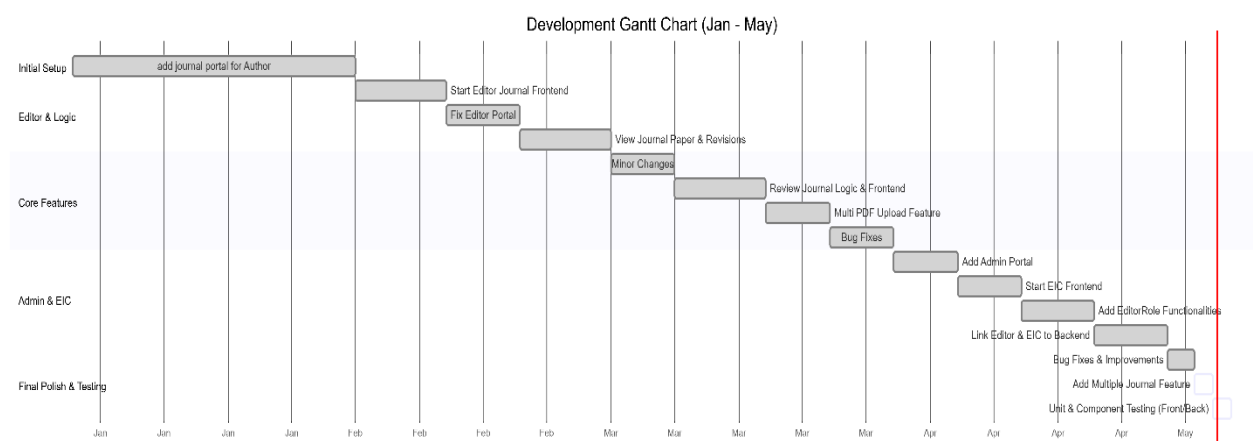


Figure-1.1: Timeline

1.5 Scope

The scope of the SubmitEase project focuses on delivering a functional, secure, and user-friendly prototype for journal and conference management.

In-Scope Features:

- **User Authentication & RBAC:** Secure login and registration with distinct, isolated dashboards for three primary roles: Authors, Reviewers, and Editors.
- **Author Module:** Capabilities for authors to submit manuscripts, upload files, and track the real-time status of their papers.
- **Editor Module:** A centralized dashboard allowing editors to view all submissions, assign specific manuscripts to peer reviewers, and record final publication decisions.
- **Reviewer Module:** Interfaces for reviewers to accept/reject review requests, access assigned manuscripts securely, and submit structured feedback.
- **Database Management:** Implementation of a robust, relational database architecture using Prisma ORM to ensure data consistency.

Out-of-Scope (Future Enhancements):

- Integration of automated AI-based plagiarism detection tools.
- Integration of a payment gateway for processing conference registration or journal publication fees.
- Automated reviewer matching algorithms based on Natural Language Processing (NLP) of the manuscript's abstract.

CHAPTER-2

LITERATURE SURVEY

2.1 Overview of Existing Work

The domain of academic submission management has evolved significantly, yet it remains fragmented between event-based conferences and continuous journal publications. A review of established systems highlights the strengths and fundamental vulnerabilities present in today's ecosystem.

2.1.1 Conference Management Systems

- **EasyChair:** One of the most globally adopted platforms, offering a comprehensive suite for paper submission, reviewer assignment, and conflict-of-interest detection. While extremely robust for large conferences, its outdated, non-responsive user interface poses a steep learning curve for new users. Customization is rigid, and the underlying technology stack often struggles with performance latency during peak submission windows.
- **Microsoft CMT (Conference Management Toolkit):** A highly scalable platform developed by Microsoft Research. It excels in managing large-scale conferences with multi-track support. However, it suffers from a lack of effective filtering options and untimely paper status updates, often leaving authors with no visibility into the review process.
- **EDAS:** Known for its advanced features like robust review matching and integration with IEEE publication services. Despite its reliability, its extensive configuration options make it overly complex for smaller conferences, and the multi-step login process hinders the user experience.

2.1.2 Academic Journal Management Systems

- **Open Journal Systems (OJS):** A widely used open-source software application for managing and publishing scholarly journals. While cost-effective and highly flexible, OJS often suffers from administrative redundancy, a clunky interface, and data synchronization complexities. It requires significant technical resources to maintain and integrate with other institutional systems.
- **ScholarOne Manuscripts:** A premier commercial system provided by Clarivate, used by top-tier publishers. It offers robust reporting, automated reviewer searching, and plagiarism checks. However, it is an expensive, closed ecosystem with a highly complex configuration process that typically requires dedicated support personnel.
- **Editorial Manager:** Widely utilized by publishers like Elsevier and Springer, this system offers highly customizable peer-review workflows. Its primary limitation is the overwhelming interface for first-time authors and reviewers. Furthermore, it operates strictly on a continuous publication model and lacks the tools necessary to simultaneously manage time-bound conference events.

2.2 Summary Table

The following table summarizes the comparison between traditional manual methods, commercial enterprise systems for conferences and journals, and the proposed SubmitEase platform.

Table-2.1: Summary Table

Feature / Capability	Enterprise Conference Systems (EasyChair, CMT)	Enterprise Journal Systems (OJS, ScholarOne)	SubmitEase (Proposed)
Workflow Integration	Strictly event-based	Strictly continuous publication	Unified (Conference & Journal)
User Experience (UX)	Dated, complex navigation	Steep learning curve, rigid	Modern, intuitive,SPA (React)
Setup & Customization	Moderate to high effort	High effort (requires IT support)	Low effort, automated host dashboard
Performance Architecture	Standard monolithic databases	Varies (OJS is often locally hosted)	Cloud-native, Edge Caching, Connection Pooling
Real-Time Status Visibility	Delayed/Limited	Moderate	High (Real-time Author Dashboards)

2.3 Limitations of Existing Methods

Several critical gaps have been identified in the current ecosystem that SubmitEase aims to address:

- **Fragmentation of Ecosystems:** Currently, academic institutions are forced to adopt disjointed systems—using platforms like EasyChair exclusively for conferences and separate platforms like OJS or ScholarOne for journal publications. There is a distinct lack of a unified platform capable of seamlessly handling both fixed-deadline events and rolling-submission journals.
- **Poor User Experience and Transparency:** Legacy platforms consistently suffer from outdated, non-responsive interfaces. Authors are often treated as secondary users, facing a "black box" review process where they cannot track their manuscript's progression in real-time.
- **Performance Bottlenecks:** Traditional database architectures struggle under the load of thousands of concurrent users hitting a hard submission deadline. Existing systems rarely leverage modern serverless or distributed concepts. SubmitEase addresses this architectural flaw by integrating Prisma Accelerate, utilizing edge caching and connection pooling to eliminate database strain and ensure high availability globally.

CHAPTER-3

SYSTEM DESIGN & METHODOLOGY

3.1 Proposed Methodology

The proposed methodology for the SubmitEase project is based on the Agile Software Development Methodology. This framework is designed to support the system's complex architecture through an iterative and incremental development process.

Key Components of the Methodology:

- **Iterative Sprints:** The project is developed in short, manageable cycles known as sprints, allowing for continuous refinement of the platform.
- **Continuous Testing and Adaptability:** This approach facilitates ongoing testing and the ability to adapt to changes in database schemas, backend logic, and caching mechanisms.
- **Structured Data Flow Management:** The methodology ensures efficient handling of the technical stack, where:
 - Client requests are processed via a Node.js and Express backend.
 - Prisma ORM serves as the bridge between the backend and the database.

3.1.1 System Architecture

Data Flow Diagram

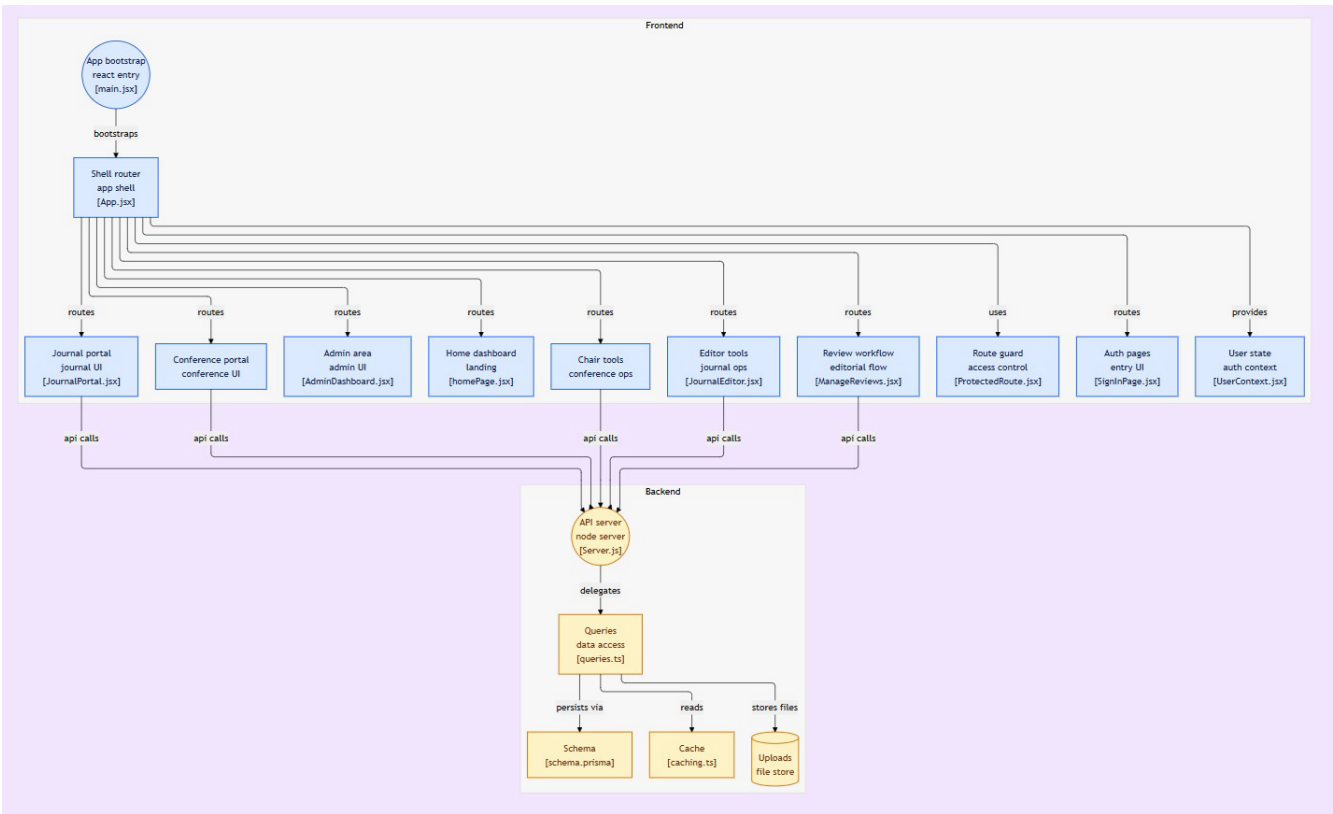


Figure-3.1: Data Flow Diagram

Use Case Diagram

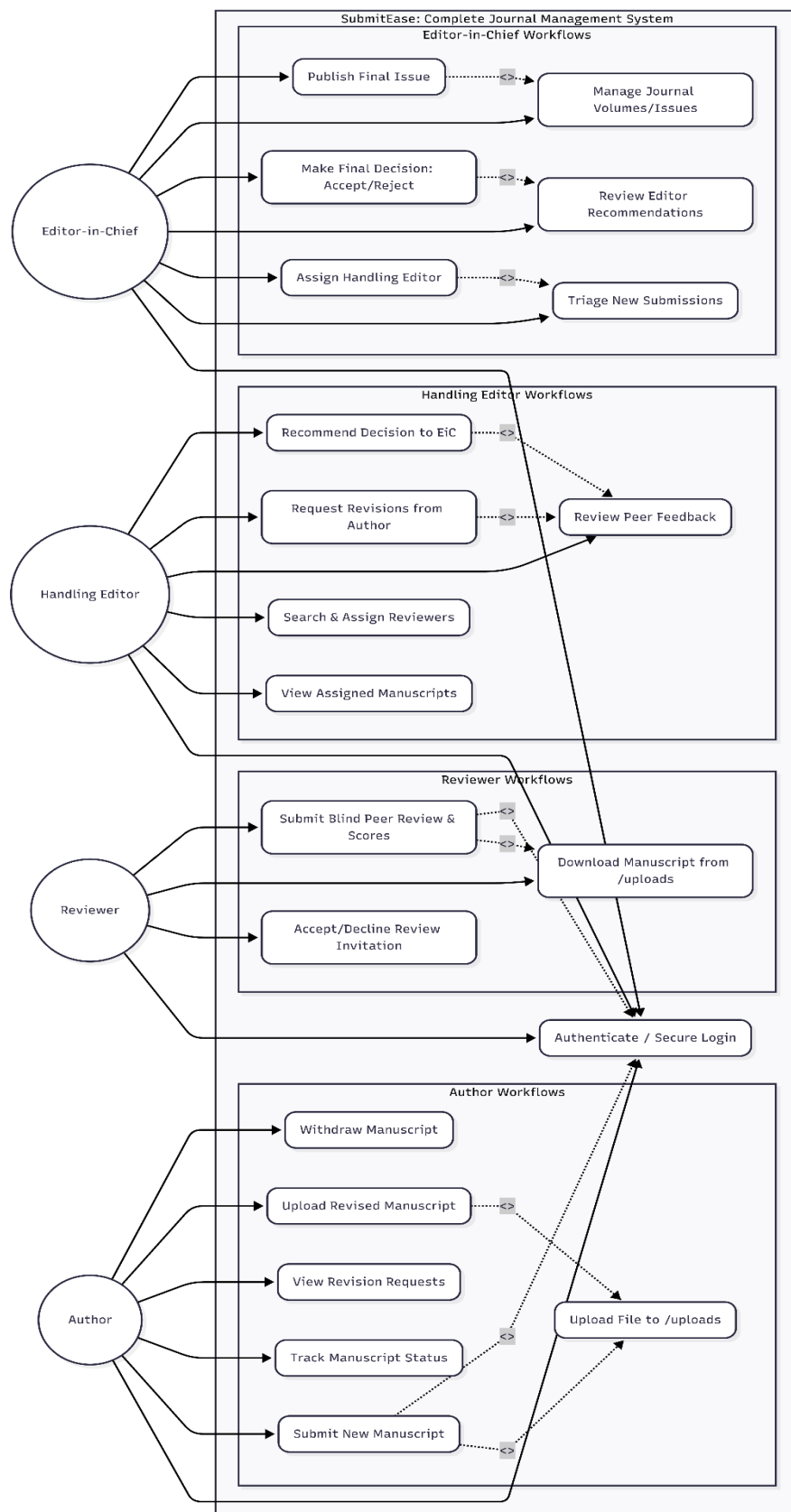


Figure-3.2: Use Case Diagram

Sequence Diagram:

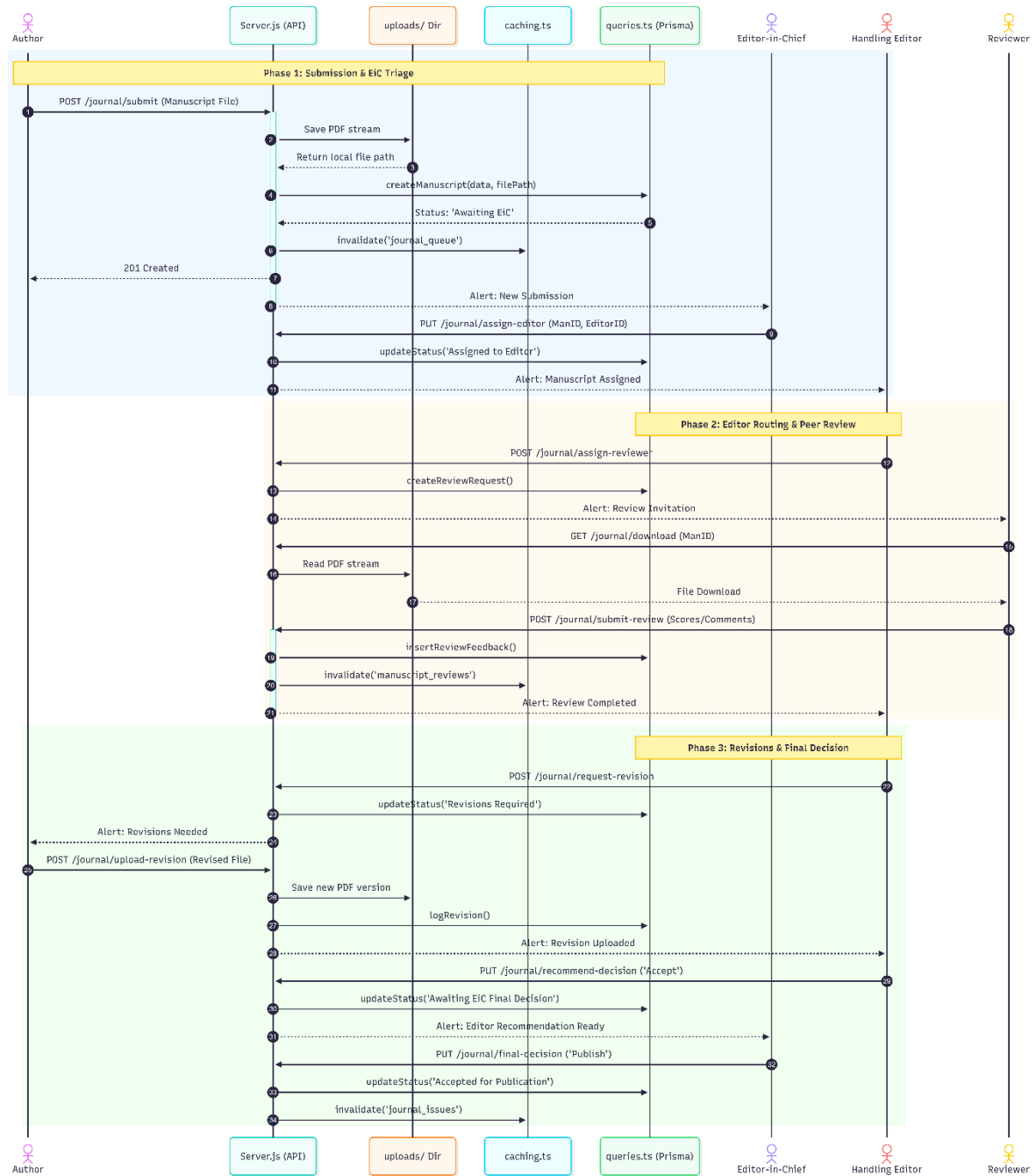


Figure-3.3: Sequence Diagram

Database Design (ER Diagram):

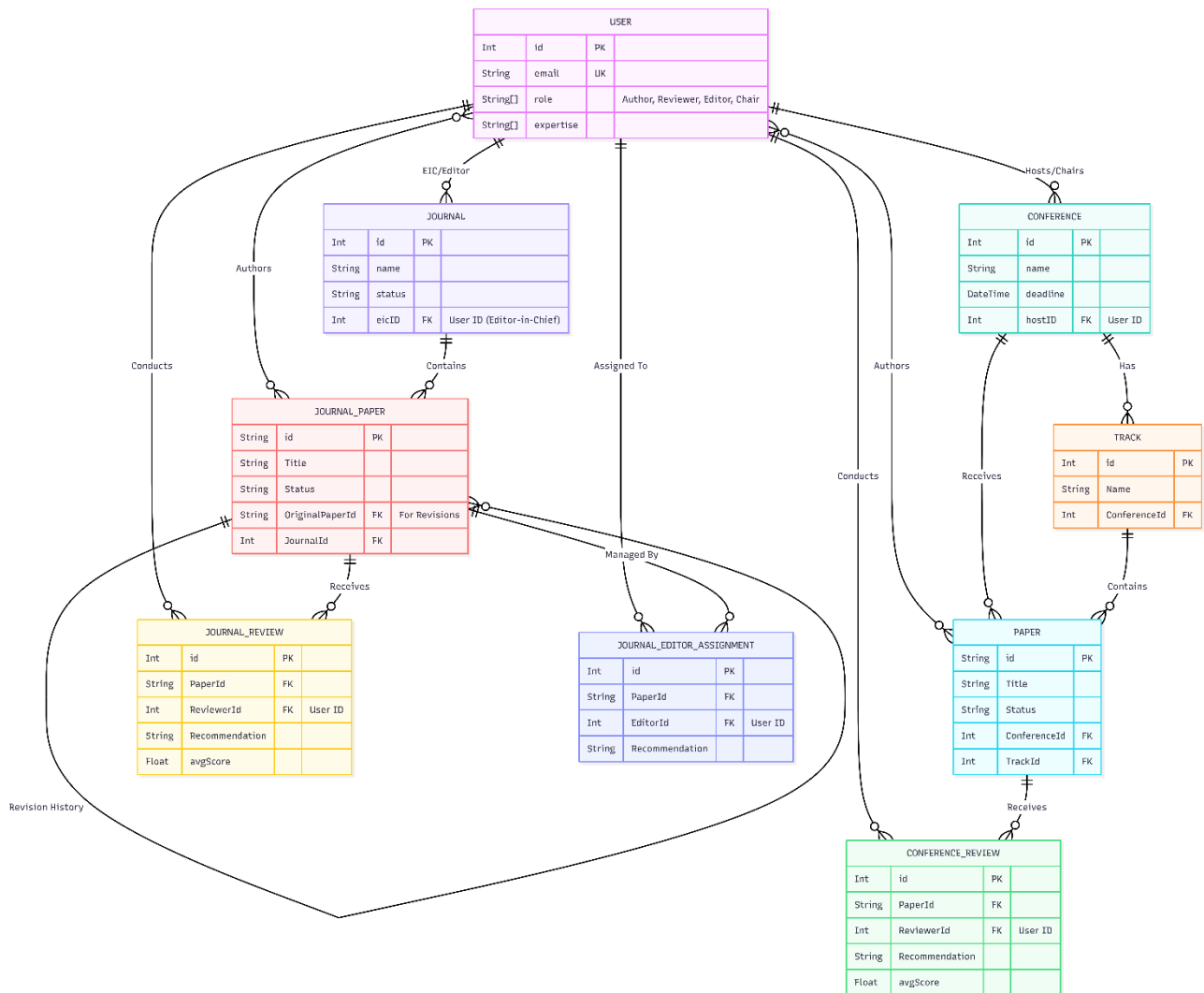


Figure-3.4: Database Design (ER-Diagram)

3.1.2 System Features

- **Author Features**

- **Profile & Expertise Management:** Authors can create profiles and update their specific expertise tags to facilitate accurate research routing.
- **Manuscript Submission Pipeline:** Authors can securely upload their research papers (PDFs up to 50MB) and define critical metadata, including title, abstract, keywords, and co-authors.
- **Automated ID Generation:** Upon submission, authors receive an automated confirmation email containing a unique, human-readable Paper ID (e.g., IJAA_26_P0001).
- **Save Draft Functionality:** Authors can save drafts of their submissions and complete the process at a later time.
- **Real-Time Status Tracking:** Authors can monitor the live progression of their manuscript through the peer-review pipeline (e.g., Under Review, Revision Required, Accepted).
- **Revision Management:** Authors can submit revised drafts seamlessly in response to editorial feedback, automatically linking the revision to the original submission history.

- **Editor Features**

- **Reviewer Invitation:** Editors can invite dedicated domain experts to rigorously evaluate the manuscript's academic validity.
- **Workflow Monitoring:** Editors can track the progress of assigned reviews and ensure deadlines are met via automated email reminders.
- **Preliminary Recommendation:** Editors can review the aggregated scores and comments from reviewers to make a preliminary recommendation to the Editor-in-Chief.

- **Reviewer Features**

- **Invitation Management:** Reviewers can accept or decline review invitations for specific papers based on their availability and domain expertise.
- **Multidimensional Scoring System:** Reviewers evaluate manuscripts using a structured 5-point sliding scale across five distinct criteria: Originality, Clarity, Soundness, Significance, and Relevance.
- **Qualitative Feedback:** Reviewers can submit detailed textual comments justifying their scores and offering constructive feedback.
- **Save Review Drafts:** Reviewers can save a draft of their evaluation before submitting the final score and recommendation (e.g., Accept, Major / Minor Revision Required, Reject).

- **Editor-in-Chief (EiC) Features**

- **Ultimate Decision Authority:** The EiC possesses the final approval authority to accept or reject journal submissions based on the aggregated evaluations.
- **Comprehensive Triage Dashboard:** The EiC can view all submitted papers, filter them by status (Under Review, Revisions Required, Accepted, Rejected), and monitor the overall acceptance rate.

- **Journal Host Features**

- **Journal Registration:** Hosts can submit requests to create a new journal by providing the journal name, official website link, publishing organization, ISSN, domain, and focus keywords.
- **EiC Assignment:** The host can search the platform's user base to assign, upgrade, or change the Editor-in-Chief responsible for the journal's editorial duties.
- **Academic Statistics Dashboard:** Hosts have access to a high-level analytics dashboard displaying total official submissions, active reviews, accepted manuscripts, and platform selectivity metrics.
- **Bulk Proceedings Download:** The Host can select multiple accepted manuscripts and generate a downloadable ZIP file containing the secure PDFs for final publication.

- **System Admin Features**

- **Platform Approval Workflow:** Admins can review pending journal and conference requests submitted by users, with the authority to approve or reject them based on platform standards.
- **Global Ecosystem Statistics:** Admins can monitor system-wide metrics, including total registered users, active journals, hosted conferences, and the total volume of papers processed.

3.2 Tool and Technique Selection

3.2.1 Frontend Technologies

- **React 19:**

- React is the industry standard for component-based user interfaces. Version 19 introduces significant improvements like the React Compiler (which automates memoization) and new hooks for forms and state (`useActionState`, `useFormStatus`). This reduces boilerplate code and drastically improves frontend performance.

- **Tailwind CSS v4:**

- Tailwind is a utility-first CSS framework. It allows developers to style components directly in the markup, leading to faster development and highly consistent design systems. Version 4 includes a new, significantly faster build engine and a zero-configuration setup, improving Developer Experience (DX).

- **Lucide React:**

- Lucide provides a consistent, clean, open-source library of SVG icons. By using the React-specific package, the project benefits from tree-shaking (only importing the icons actually used, keeping the bundle size small) while seamlessly integrating with React components.

3.2.2 Backend Technologies

- **Node.js v22:**
 - Node.js allows for a unified JavaScript/TypeScript stack across both the client and server. Version 22 is an Active LTS (Long Term Support) release, providing critical stability, security updates, and performance enhancements like a built-in test runner and native WebSocket support.
- **Express v5:**
 - Express is a fast, unopinionated, minimalist web framework. Version 5 introduces native Promise support. This is a major upgrade, as it allows developers to write asynchronous route handlers without needing repetitive try/catch blocks or external wrapper libraries for error handling.

3.2.3 Database and Storage

- **Prisma Postgres:**
 - Prisma Postgres manages our complex data structures (users, journals, submissions) by combining the robust reliability of PostgreSQL with Prisma's integrated, managed ORM service. This provides an intuitive, maintainable schema and strict type-safety, ensuring database errors are caught early in development rather than crashing the application in production.
- **Supabase:**
 - Supabase acts as an open-source Backend-as-a-Service (BaaS) built on top of Postgres. It is typically chosen to rapidly implement managed PostgreSQL hosting, out-of-the-box authentication, and scalable file storage (e.g., for storing PDF paper submissions), reducing infrastructure overhead.

3.2.4 Build Tools and Version Control

- **Vite:**
 - Vite is a next-generation frontend build tool. It replaces older tools like Webpack because it offers near-instant server start times and incredibly fast Hot Module Replacement (HMR). This means when a developer saves a file, the browser updates instantly, massively accelerating the development cycle.
- **ESLint:**
 - ESLint is a static code analysis tool. It is crucial for enforcing coding standards, maintaining consistency across a team, and catching syntax errors or problematic patterns before the code is even run.
- **Git:**
 - Git is the universally required standard for source code version control. It allows developers to track changes, work collaboratively on different features via branches, and safely manage the project's history.

CHAPTER-4

IMPLEMENTATION AND TESTING

4.1 Experimental Setup

This section details the hardware and software environment utilized to develop, test, and deploy the SubmitEase platform.

- **Hardware Environment:**
 - **Processor:** Intel Core i5/i7 (or AMD equivalent / Apple Silicon)
 - **RAM:** 8 GB / 16 GB
 - **Storage:** 256 GB / 512 GB SSD
 - **Operating System:** Windows 11 / macOS / Ubuntu Linux
 - **Browser:** Google Chrome / Microsoft Edge / Firefox
- **Software & Technology Stack:**
 - **Frontend Environment:**
 - **Library:** React.js (v19)
 - **Build Tool:** Vite
 - **Styling:** Tailwind CSS v4
 - **Icons:** Lucide React
 - **Backend Environment:**
 - **Runtime:** Node.js (v22)
 - **Framework:** Express.js (v5)
 - **Database & Storage:**
 - **Database:** Prisma Postgres (Relational DB)
 - **ORM & Middleware:** Prisma Client with Prisma Accelerate (for connection pooling and global edge caching)
 - **Object Storage:** Supabase Storage (for hosting PDF manuscript files)

- **Development Tools:**
 - **IDE:** Visual Studio Code (VS Code)
 - **Version Control:** Git and GitHub
 - **API Testing:** Postman
 - **Package Manager:** npm / yarn

4.2 Test Cases / Implementation Outputs

4.2.1 Testing Implementation

The testing methodology for SubmitEase follows a structured, multi-tier approach to ensure code reliability, data integrity, and a seamless user experience. The strategy encompasses Unit Testing, Integration Testing, and End-to-End (E2E) feature testing.

- **Backend Unit & API Testing (Node.js/Express):**
 - **Objective:** To validate the core business logic and individual API endpoints in isolation.
 - **Tools Used:** Jest and Supertest (standard for Node.js environments).
 - **Focus Areas:**
 - Testing the createAbbreviation and Paper ID Generation algorithms to ensure they increment correctly (e.g., transitioning from P009 to P010).
 - Validating the Review Scoring Logic to ensure accurate calculation of averages.
 - API endpoint testing (e.g., ensuring POST /login returns a 200 OK with a valid JWT token on success, and a 401 Unauthorized on failure).
- **Frontend Unit & Component Testing (React):**
 - **Objective:** To ensure individual UI components render correctly and handle local state updates without errors.
 - **Tools Used:** React Testing Library and Jest.
 - **Focus Areas:**
 - Validating form validations (e.g., ensuring the submission button is disabled if required fields are empty).

- Testing the Reviewer Slider components to ensure they accurately capture and bind values between 1-5.
- Verifying that conditional rendering works based on user roles (e.g., the 'Create Track' button only appears for Host users).
- **Integration & System Testing (Database & Storage)**
 - **Objective:** To verify that the backend correctly communicates with Prisma Postgres and Supabase Storage.
 - **Focus Areas:**
 - Confirming that when a PDF is uploaded via the frontend, the multer middleware correctly processes it, Supabase successfully stores it, and the resulting public URL is correctly saved into the Prisma Paper model.
 - Testing database relationships (e.g., ensuring a Review is correctly linked to both a PaperId and a ReviewerId).

```

RERUN src/tests/Login.test.jsx x7
✓ src/tests/Login.test.jsx (2 tests) 589ms
✓ SignInPage Component (2)
  ✓ renders sign-in form elements correctly 549ms
  ✓ requires email and password fields to be filled before submission 23ms
Test Files 1 passed (1)
Tests 2 passed (2)
Start at 17:06:32
Duration 1.94s
PASS Waiting for file changes...
press h to show help, press q to quit

This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act
✓ src/tests/JournalPortal.test.jsx (3 tests) 540ms
✓ JournalPortal Component (3)
  ✓ renders the main portal headers correctly 472ms
  ✓ displays all the dashboard statistic cards 32ms
  ✓ displays the document filtering tabs 28ms
Test Files 1 passed (1)
Tests 3 passed (3)
Start at 17:18:22
Duration 1.40s
PASS Waiting for file changes...
press h to show help, press q to quit

🔥 Server connected to Prisma Postgres, running at http://localhost:3001
at Server.log (Server.js:4910:11)
PASS tests/api.test.js
SubmitEase Backend API Validation Tests
✓ POST /send-verification should return 400 if email is missing (254 ms)
✓ POST /upload should return 400 if no PDF file is attached (15 ms)
✓ POST /conference/assign-publication-chairs should return 400 for missing array (16 ms)
Test Suites: 1 passed, 1 total
Tests: 3 passed, 3 total
Snapshots: 0 total
Time: 3.78 s, estimated 13 s
Ran all test suites.

```

Figure-4.1: Frontend And Backend Testing

4.2.2 Backend Implementation

Database Schema

```
model User {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  password    String
  firstname   String
  lastname    String
  role        String[]
  expertise   String[]
  organisation String
  country     String
  isAdmin     Boolean      @default(false)

  // Conference Relations
  Paper          Paper[]
  hostedConferences Conference[] @relation("HostToConference")
  reviewedConferences Conference[] @relation("ReviewerToConference")
  Reviews        Reviews[]
  ConfReviewInvitation Conference[] @relation("InvitedReviewerToConference")
  ChairedTracks  Tracks[] @relation("TrackChairs")
  PublicationChairs Conference[] @relation("PublicationChairs")
  RegistrationChairs Conference[] @relation("RegistrationChairs")

  // Journal Relations
  hostedJournals Journal[] @relation("HostToJournal")
  JournalEIC      Journal[] @relation("EICToJournal")
  JournalAssignments JournalAssignment[]
  JournalPapers   JournalPapers[] @relation("JournalAuthors")
  JournalReviews   JournalReviews[]
  JournalEditors   JournalEditors[] @relation("PrimaryEditor")
  TransferredAssignments JournalEditors[] @relation("TransferredEditor")
}

model Journal {
  id          Int          @id @default(autoincrement())
  name        String
  link        String
  status      String
  Publication String
  submittedAt DateTime
  Keywords    String[]
  hostID      Int
  host        User?        @relation("HostToJournal", fields: [hostID], references: [id])
  eicID       Int?
  EIC         User?        @relation("EICToJournal", fields: [eicID], references: [id])
  Assignments JournalAssignment[]
  Papers      JournalPapers[]
}

model JournalAssignment {
  JournalId Int
  Journal   Journal @relation(fields: [JournalId], references: [id])
  UserId    Int
  User      User    @relation(fields: [UserId], references: [id])
  role      String[] // Array of roles this specific user has in this specific journal

  @@unique([JournalId, UserId])
}
```

```

model JournalPapers {
  id          String      @id
  Title       String
  Authors     User[]      @relation("JournalAuthors")
  URL         String?
  RevisionPaperURL String?
  Abstract    String
  Keywords    String[]
  Status      String
  submittedAt DateTime
  AuthorOrder Int[]
  Reviews     JournalReviews[]
  OriginalPaperId String?
  OriginalPaper JournalPapers? @relation("PaperHistory", fields: [OriginalPaperId], references: [id], onDelete: SetNull)
  Revisions   JournalPapers[] @relation("PaperHistory")
  Editors     JournalEditors[]
  JournalId   Int
  Journal     Journal      @relation(fields: [JournalId], references: [id])
}

model JournalReviews {
  id          Int          @id @default(autoincrement())
  PaperId     String
  Paper       JournalPapers? @relation(fields: [PaperId], references: [id])
  ReviewerId  Int
  User        User?        @relation(fields: [ReviewerId], references: [id])
  Comment     String?
  Recommendation String?
  Status      String?
  isBlind     Boolean?
  submittedAt DateTime?
  assignedAt  DateTime?
  scoreOriginality Int?
  scoreClarity Int?
  scoreSoundness Int?
  scoreSignificance Int?
  scoreRelevance Int?
  avgScore    Float        @default(0)
  isVisibleToAuthor Boolean @default(false)

  @@unique([PaperId, ReviewerId])
}

model JournalEditors {
  id          Int          @id @default(autoincrement())
  PaperId     String
  Paper       JournalPapers? @relation(fields: [PaperId], references: [id])
  EditorId    Int
  Editor      User?        @relation("PrimaryEditor", fields: [EditorId], references: [id])
  Recommendation String?
  Status      String?
  submittedAt DateTime?
  assignedAt  DateTime?    @default(now())
  TransferredEditorId Int?
  TransferredEditor User?   @relation("TransferredEditor", fields: [TransferredEditorId], references: [id])
  TransferStatus String?
}

```

Figure-4.2: Database Schema

API Documentation And Endpoints

Base URL: <http://localhost:3001>

Table-4.1: API Documentation

Method	Endpoint	Description	Request Body
POST	/users	Register User	{email, password, firstname...}
POST	/login	User Login	{email, password}
POST	journal/savepaper	Submit Journal Paper	FormData {pdfFile, title, confId...}
POST	journal/submit-review	Submit Review	{paperId, scores...}
POST	journals/discover	Get All Registered Journals	{expertise}

4.2.3 Frontend Implementation

Home Page

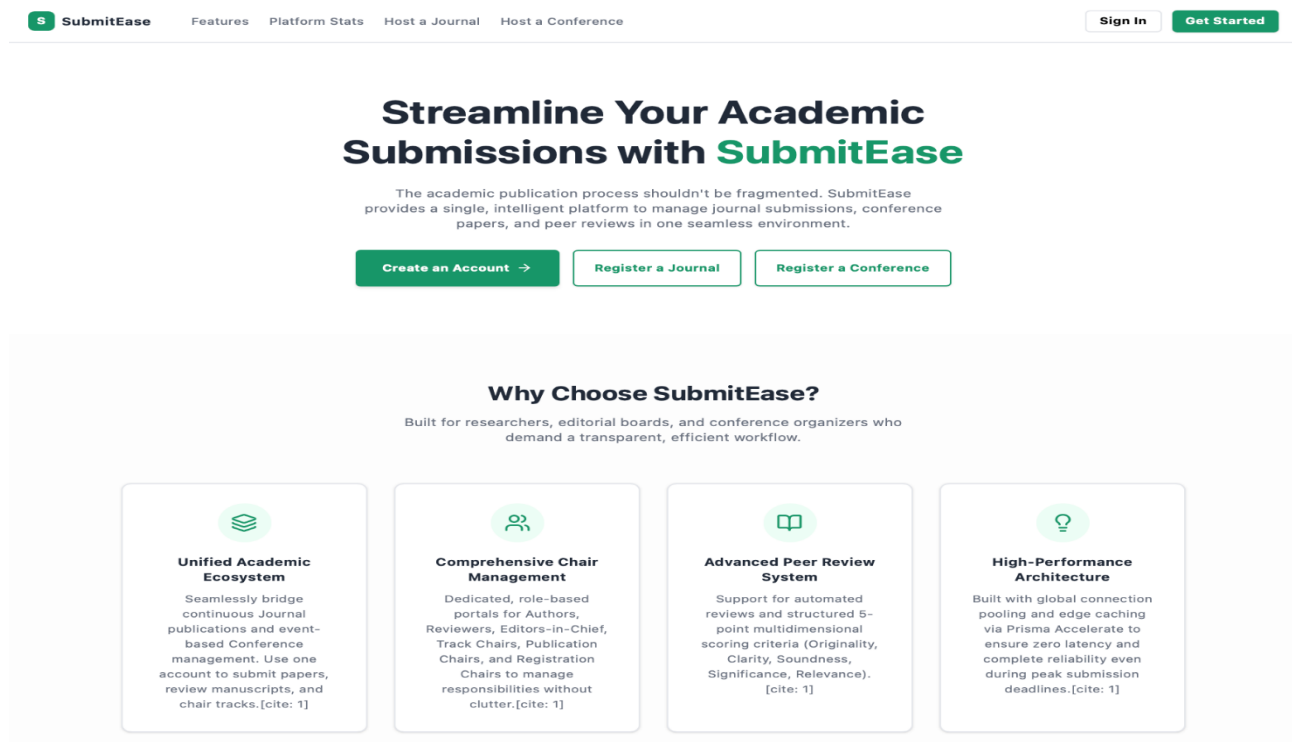


Figure-4.3: Home Page

User Dashboard

S

SubmitEase

Journal Portal

Conference Portal

Logout

Profile Information

VT

Full Name

Vasu Tayal

Organisation

JIIT

Email Address

vasutayal800@gmail.com

Research Area (Comma separated)

Data Science, Artificial Intelligence

Change Email

Edit Profile

Academic Statistics

0

Registered Conferences

2

Journals Registered

2

Total Submissions

0

Paper Accepted

Recent Activity

#IJAAA_2_P0002

Mitigating Algorithmic Bias in Generative Adversarial Networks

Status: Under Review

International Journal of Advanced AI

5/6/2026

#IJAAA_2_P0001

Transformer Models for Real-time Edge Video Processing

Status: Under Review

International Journal of Advanced AI

5/6/2026

Access Portals

Choose your portal to manage submissions, track progress, and access academic resources.

Journal Access Portal

Manage your active journals or discover new publications.

My Registered Journals

Search & Discover

Search journals by name or publisher...

★ RECOMMENDED FOR YOUR EXPERTISE

International Journal of Advanced AI

TechScience Publishers

Visit Website

Register

ALL AVAILABLE JOURNALS

Global Review of Sustainable Computing

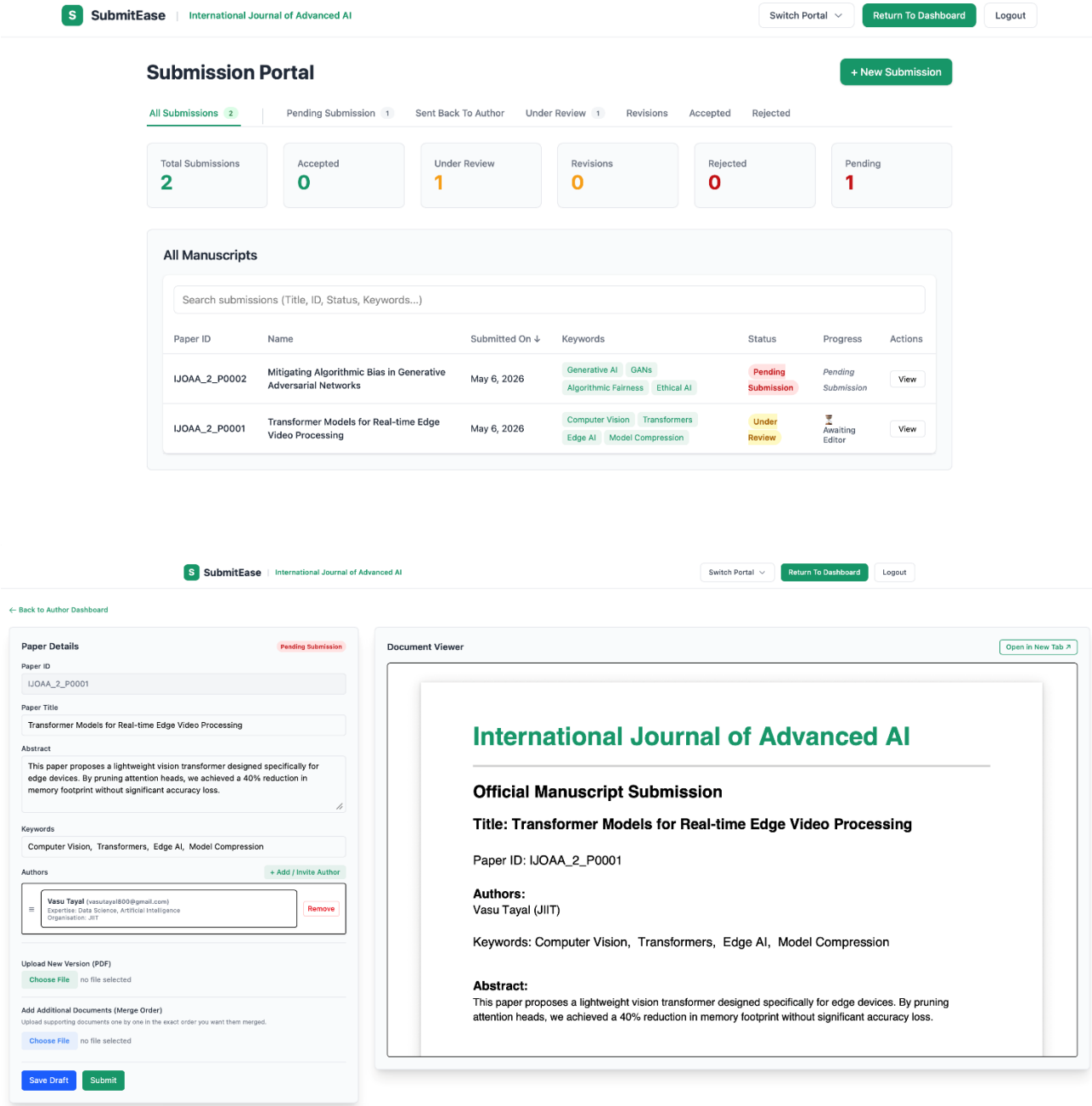
GreenEarth Academic Institute

Visit Website

Register

Figure-4.4: User Dashboard

25



S

SubmitEase

| International Journal of Advanced AI

Switch Portal

Return To Dashboard

Logout

← Back to Author Dashboard

Paper Details

Pending Submission

Paper ID

IJOAA_2_P0001

Paper Title

Transformer Models for Real-time Edge Video Processing

Abstract

This paper proposes a lightweight vision transformer designed specifically for edge devices. By pruning attention heads, we achieved a 40% reduction in memory footprint without significant accuracy loss.

Keywords

Computer Vision, Transformers, Edge AI, Model Compression

Authors

+ Add / Invite Author

Vasu Tayal (vusutayal800@gmail.com)
Expertise: Data Science, Artificial Intelligence
Organisation: JIIT

Remove

Upload New Version (PDF)

Choose File

no file selected

Add Additional Documents (Merge Order)

Upload supporting documents one by one in the exact order you want them merged.

Choose File

no file selected

Save Draft

Submit

Document Viewer

Open in New Tab ↗

International Journal of Advanced AI

Official Manuscript Submission

Title: Transformer Models for Real-time Edge Video Processing

Paper ID: IJOAA_2_P0001

Authors:

Vasu Tayal (JIIT)

Keywords: Computer Vision, Transformers, Edge AI, Model Compression

Abstract:

This paper proposes a lightweight vision transformer designed specifically for edge devices. By pruning attention heads, we achieved a 40% reduction in memory footprint without significant accuracy loss.

Figure-4.5: Author Portal

Reviewer

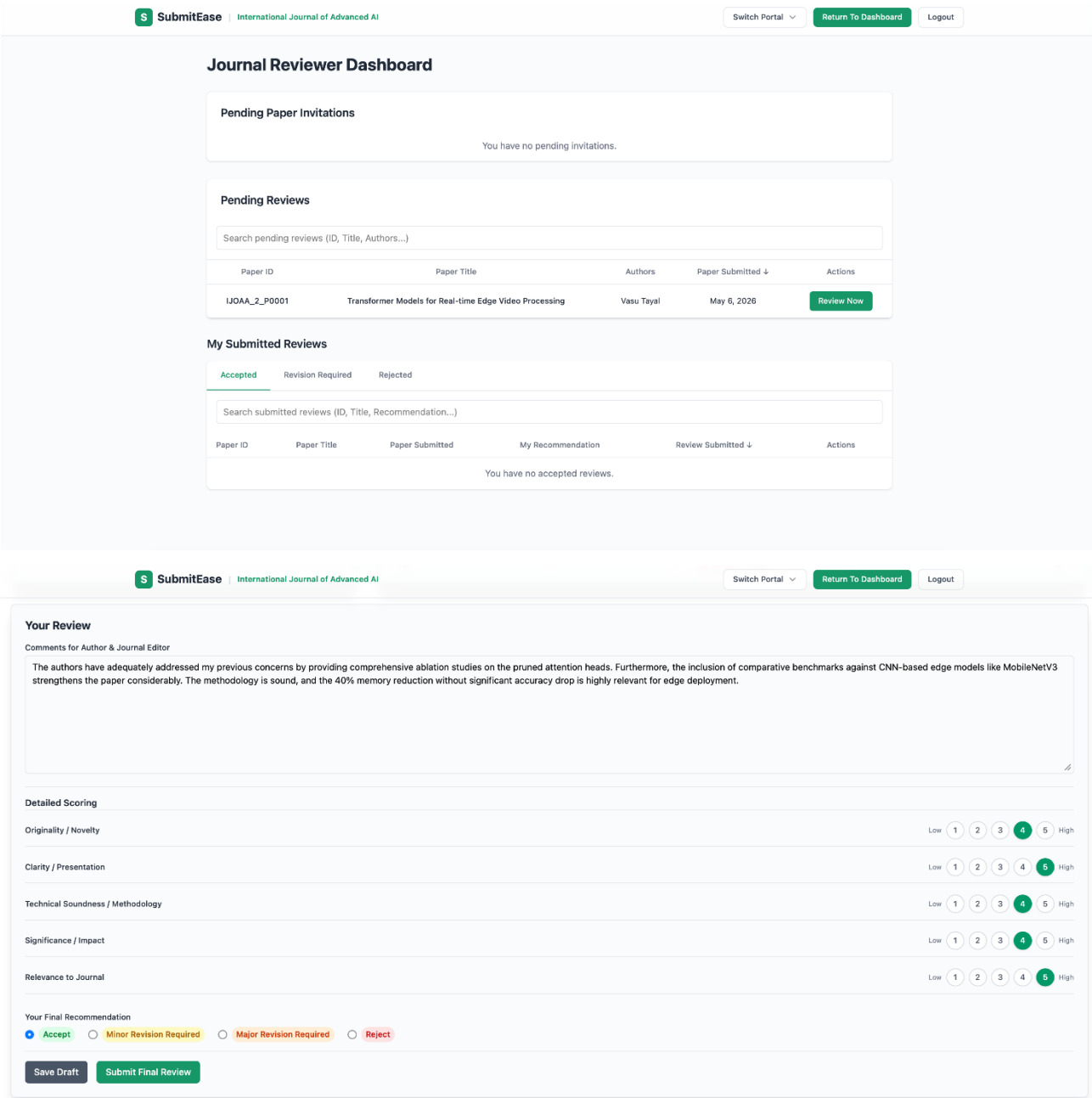


Figure-4.6: Reviewer Portal

S

SubmitEase

International Journal of Advanced AI

Switch Portal

Return To Dashboard

Logout

Editor Dashboard

All Papers1

Unassigned

Needs Reviewers1

Awaiting Decisions

Decisions Made

Transfers Pending

TOTAL RECEIVED1

UNASSIGNED0

NEEDS REVIEWERS1

IN REVIEW0

DECISIONS MADE0

TRANSFERS PENDING0

All Manuscripts

Search papers (Title, Author, ID, Keywords...)

ID	Paper Details	Reviewers	Date ↓	Status	Actions
IJOAA_2_P0001	Transformer Models for Real-time Edge Video Processing Unknown Author	Invited: 1 Accepted: 1	May 6, 2026	Under Review	View

Reviewer Management

Assign or Invite Reviewer

Select	Reviewer	Status
☐	Ashutosh Tiwari aadityapratapcr7@gmail.com	Submitted

Submit Editor Recommendation

You must have at least 3 completed reviews submitted for this paper before you can send your formal recommendation to the Editor-in-Chief.

Accept

Minor Revision

Major Revision

Send Recommendation to EIC

Detailed Review Submission

REVIEWER INFORMATION

Ashutosh Tiwari
aadityapratapcr7@gmail.com

STATUS & RECOMMENDATION

Submitted

Accept

EVALUATION METRICS (OUT OF 10)

Originality 4	Clarity 5	Soundness 4
Significance 4	Relevance 5	Average Score 4.4

WRITTEN FEEDBACK

Inclusion of comparative benchmarks against CNN-based edge models like MobileNetV3 strengthens the paper considerably. The methodology is sound, and the 40% memory reduction without significant accuracy drop is highly relevant for edge deployment.

Close Window

Reviewer Management

Assign or Invite Reviewer

Select	Reviewer	Status	Recommendation & Comments	Assigned ↓	Days Pending	Actions
☐	Ashutosh Tiwari aadityapratapcr7@gmail.com	Submitted	Accept Inclusion of comparative benchmarks against CNN-based edge models like MobileNetV3 strengthens the paper considerably. The methodology is sound...	May 6, 2026	—	View Details

Submit Editor Recommendation

You must have at least 3 completed reviews submitted for this paper before you can send your formal recommendation to the Editor-in-Chief.

Accept

Minor Revision

Major Revision

Reject

Send Recommendation to EIC

Figure-4.7: Editor Portal

28

Journal Host

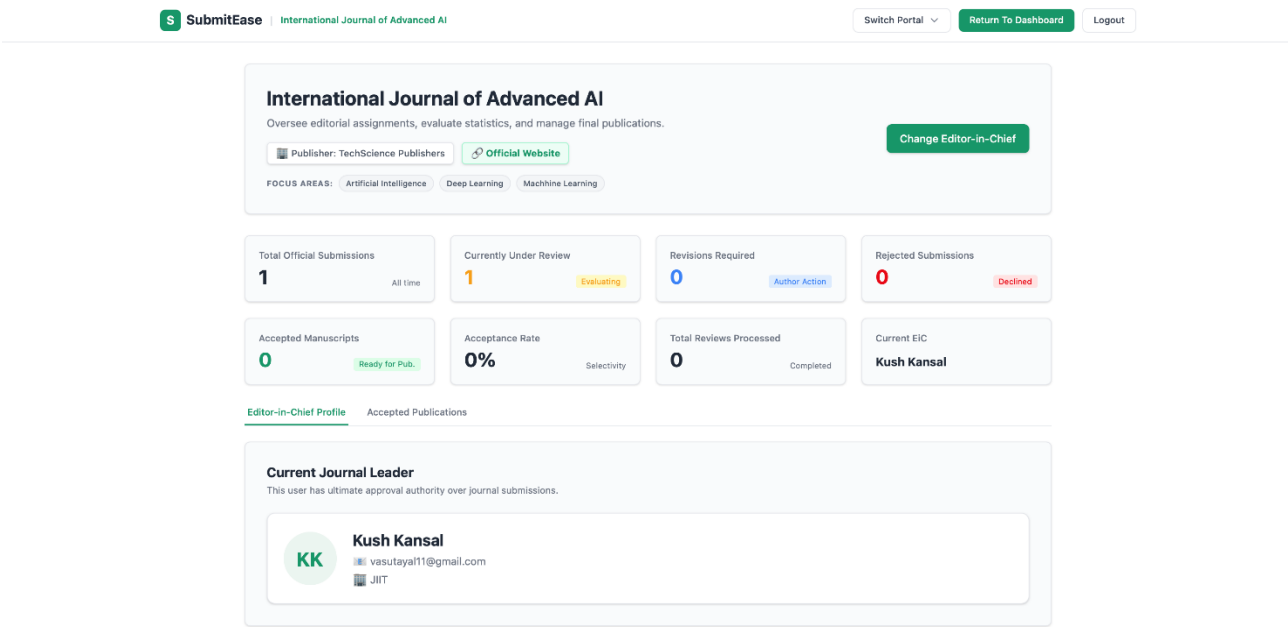


Figure-4.8: Journal Host Portal

Editor-in-Chief

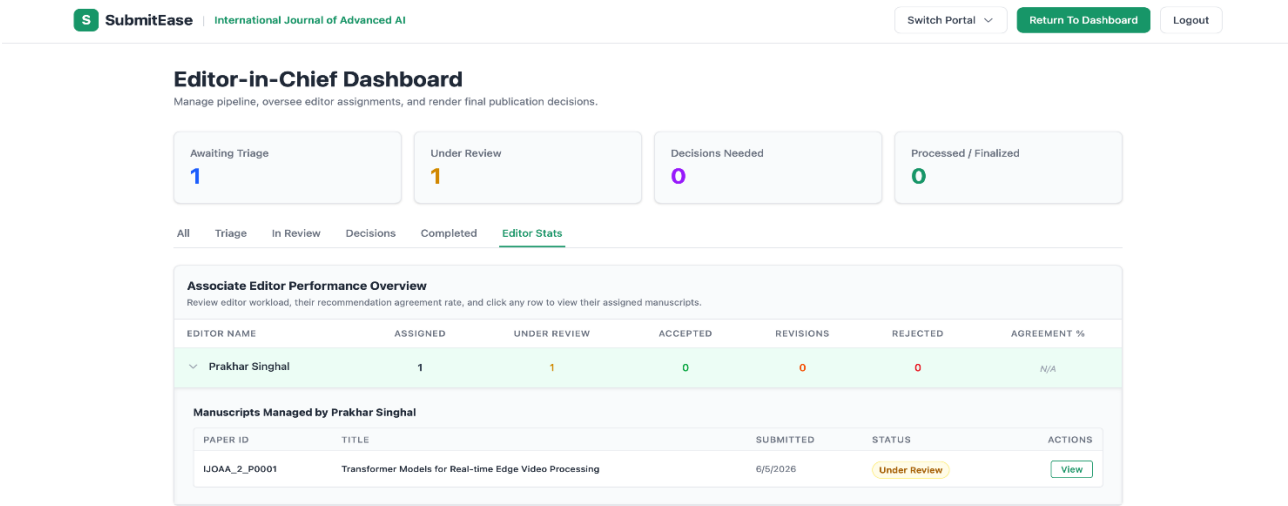


Figure-4.9: EIC Portal

Admin

S

SubmitEase Admin

Conference Admin

Journal Admin

Logout

Platform Overview

Monitor system health, recent activity, and platform growth.

System Status: Online

Total Conferences20 Pending

Registered Journals22 Pending

Registered Users8Platform Wide

Journal Submissions00 Accepted

Conference Papers21 Accepted

Recently Created ConferencesView All →

ID	CONFERENCE NAME	STARTS AT	ACTIONS
#2	International Conference on Computer Engineering Closed	Mar 31, 2026	View Details
#1	International Conference on Computer Science Closed	Apr 1, 2026	View Details

Recently Registered JournalsView All →

ID	JOURNAL NAME	PUBLISHER	ACTIONS
#3	Global Review of Sustainable Computing Pending	GreenEarth Academic Institute	View Details
#2	International Journal of Advanced AI Pending	TechScience Publishers	View Details

Conference Admin

Manage pending requests, event tracks, and organizers.

Journal Admin

Oversee Editor-in-Chief roles and final publications.

S

SubmitEase Admin

Conference Admin

Journal Admin

Return To Dashboard

Logout

Journal Operations

Manage journal approvals, monitor editorial boards, and analyze platform-wide submission volumes.

Total Journals22 Pending

Active Journals0Accepting

Total Papers Hosted0Submissions

Avg. Papers / Journal0Density

All Journal Data

Pending Approvals 2

Search journals by name or publisher...

Journal ↑	Host / Publisher	Actions
Global Review of Sustainable Computing GreenEarth Academic Institute	Aanya Varshney aanyavarshney0530@gmail.com	View Details Approve Reject
International Journal of Advanced AI TechScience Publishers	Arpit Varshney arpitvarshney5@gmail.com	View Details Approve Reject

S

SubmitEase Admin

Conference Admin

Journal Admin

Return To Dashboard

Logout

Journal Operations

Manage journal approvals

Total Journals2

All Journal Data

Search journals by

Journal ↑

Global Review of S

GreenEarth Academic

International Journ

TechScience Publish

Journal Details

Global Review of Sustainable Computing

GreenEarth Academic Institute

Visit Journal Website

Pending

Reject

Approve

PUBLISHER

GreenEarth Academic Institute

HOST USER

Aanya Varshney

PLATFORM STATUS

Pending

SUBMITTED ON

May 6, 2026

EDITORIAL ADMINISTRATION

JOURNAL HOST

Aanya Varshney

aanyavarshney0530@gmail.com

JIT

EDITOR-IN-CHIEF

Not Yet Assigned

Figure-4.10: Admin Portal

CHAPTER-5

RESULTS & DISCUSSION

5.1 Result analysis

- **Workflow Validation:** The core submission workflows for journal modules were successfully executed and validated. For the journal module, the recursive revision loop was thoroughly tested. The system successfully linked multiple manuscript versions (e.g., initial submission, R1, R2) back to their original PaperId within the PostgreSQL database. This allowed the editorial dashboard to display the complete lineage of a submission without duplicating static metadata, confirming the robustness of the relational schema.
- **Role-Based Access Control (RBAC) Results:** The implementation of Role-Based Access Control (RBAC) was proven highly effective in maintaining academic confidentiality. Security protocols successfully isolated user permissions across the platform. Tests confirmed that authors were completely restricted from viewing peer review scores, reviewer comments, or reviewer identities until the Editor-in-Chief officially recorded and published a final decision. Furthermore, reviewers could only query and download manuscripts that were explicitly assigned to them by the handling editor. This strict data isolation successfully demonstrated the system's capacity to uphold a secure, blind peer review process.
- **Storage & Metadata Integration:** The integration between the cloud storage bucket and the relational database performed flawlessly. When authors uploaded manuscript files, the system securely deposited the PDFs into Supabase storage and instantly retrieved the secure public URLs. These URLs, alongside the structured metadata (including titles, abstracts, keywords, and co-author arrays), were synchronously written to the Prisma-managed PostgreSQL database. Retrieval tests confirmed that the database queries correctly mapped to the hosted files with minimal latency, ensuring no instances of orphaned files or corrupted metadata links during the submission phase.

5.2 Performance Evaluation and Comparison with Existing Methods

5.2.1 Performance Evaluation

The performance of the SubmitEase platform was evaluated based on system responsiveness, database transaction efficiency, and file storage handling. The architecture, built on the PERN stack (PostgreSQL, Express.js, React, Node.js) and integrated with Supabase, demonstrated high reliability under simulated workloads.

- **API Responsiveness:** Backend routing was handled via Express.js. During testing, standard CRUD operations (such as fetching user dashboards or updating manuscript metadata) were processed reliably without any request timeouts or server errors.
- **Database Query Efficiency:** By utilizing Prisma ORM, complex relational queries—such as fetching the recursive, multi-version history of a journal manuscript alongside its peer review feedback—were optimized. The system efficiently handles deeply nested joins without significant latency.
- **Storage and File Handling:** Manuscript files (PDF) were managed using Supabase storage buckets. File uploads demonstrated high success rates and secure, token-based retrieval.
- **Scalability & Concurrency:** The stateless JWT-based authentication ensures that the backend remains lightweight. The separation of binary file storage (Supabase) from structured metadata (PostgreSQL) prevents database bloat, allowing the system to scale efficiently as the number of submissions and revisions grows.

5.2.2 Comparison with Existing Methods

Academic publishing typically relies either on heavily fragmented legacy software (e.g., EasyChair, Editorial Manager) or manual, email-based submission tracking. SubmitEase was designed to address the specific bottlenecks found in these traditional approaches.

1. Unified Architecture vs. Fragmented Systems:

Existing platforms usually cater strictly to either Conferences (fixed deadlines, linear progression) or Journals (rolling deadlines, recursive revisions). SubmitEase unifies both paradigms into a single platform. This significantly reduces the learning curve for researchers and institutions who previously had to purchase, manage, and learn multiple separate systems.

2. Automated Workflow vs. Manual Tracking:

Many smaller, institution-level conferences still rely on email threads and spreadsheets to track submissions, assign reviewers, and calculate scores. SubmitEase completely eliminates manual triage by introducing automated deadline lockouts and an algorithmic dashboard that instantly aggregates peer-review scores for the Program Chair.

3. Modern UI/UX vs. Legacy Interfaces:

Traditional academic systems are notorious for outdated, clunky user interfaces that lead to formatting errors and submission fatigue. SubmitEase leverages React to provide a modern, responsive, and role-specific dashboard. Authors, Reviewers, and Editors only see the data and actions relevant to their specific permissions, drastically reducing cognitive load.

Figure-5.1: Comparison Table

Feature / Metric	Legacy Conference Systems (e.g., EasyChair)	Legacy Journal Systems (e.g., Editorial Manager)	SubmitEase (Unified Platform)
System Architecture	Segmented, often built specifically for single-event lifecycle	Complex, recursive workflow logic.	Unified PERN Stack; modular design handles both linear and recursive flows.
Workflow Timing	Strictly linear; fixed submission and review windows.	Continuous, rolling submissions; no fixed volume dates initially.	Dual-Track Support: Handles both fixed deadlines and rolling continuous submissions within the same infrastructure.
Peer Review Process	Single-round or two-round binary decisions (Accept/Reject).	Multi-round, recursive loops (Major/Minor Revisions).	Flexible Assignment: Adapts to linear (conference) or recursive (journal) grading scales seamlessly.
Version Control	Typically limited to an initial upload and a final "Camera-Ready" upload.	Robust; tracks multiple rounds of author revisions.	High Versioning: Self-referencing database links track the complete lineage of any manuscript.
User Interface (UI)	Often outdated, with a high learning curve for new researchers.	Highly dense, text-heavy interfaces; steep learning curve.	Modern, Responsive (React): Clean, role-based dashboards that reduce cognitive load.

CHAPTER-6

CONCLUSION AND FUTURE SCOPE

6.1 Summary of Work

The development of SubmitEase successfully addressed the core inefficiencies present in traditional academic publishing and conference management. By engineering a unified platform utilizing the PERN stack (PostgreSQL, Express.js, React, Node.js) alongside Prisma ORM and Supabase, the project achieved a robust separation of concerns while maintaining a seamless user experience.

The primary technical achievement of this work is the dual-track architectural design, which natively accommodates both the linear, deadline-driven progression of academic conferences and the recursive, continuous revision loops of academic journals. The implementation of a strict Role-Based Access Control (RBAC) system ensures secure and distinct workflows for Authors, Reviewers, Editors, and Program Chairs. Furthermore, the database schema successfully tracks manuscript version histories without data duplication, providing a highly scalable solution for institutional use.

6.2 Impact on society, Environmental sustainability, Ethical issues and compliance

- **Societal and Industry Impact:** SubmitEase democratizes the academic peer-review process. By providing a modern, low-cost, and unified management platform, smaller universities and independent research organizations can host rigorous academic conferences without relying on expensive legacy software. This accelerates the dissemination of scientific knowledge by reducing administrative bottlenecks.
- **Environmental Sustainability:** The platform champions a strictly paperless workflow. By handling all manuscript uploads, peer reviews, and editorial decisions digitally via cloud infrastructure, SubmitEase significantly reduces the carbon footprint associated with printing, mailing, and physically storing academic proceedings.
- **Ethical Issues and Compliance:** Academic integrity is central to this platform. The RBAC system is specifically designed to enforce double-blind peer review standards, ensuring that reviewers cannot access author identities and vice versa, thereby mitigating unconscious bias. Data privacy is maintained through JWT authentication and secure, token-based database access, ensuring compliance with modern data protection standards regarding intellectual property.

6.3 Limitations of the Work

While SubmitEase provides a comprehensive baseline for manuscript management, the current iteration has a few operational limitations:

- **File Format Constraints:** The system currently relies on the manual upload of standard document formats (PDF/DOCX). It does not yet feature an in-browser rich-text editor for direct manuscript formatting.
- **External Integrations:** The platform operates as a standalone system and is not currently integrated with external academic databases (such as ORCID or Crossref) for automated author verification or metadata fetching.
- **Manual Integrity Checks:** While the system handles the routing of papers flawlessly, initial desk checks for plagiarism or incorrect formatting still require manual oversight by the Editor-in-Chief or Program Chair.
- **Notification Channels:** Automated notifications are currently constrained to the platform's internal dashboard and standard email, lacking push notifications for mobile devices.

6.4 Future Scope

To further enhance the platform's utility and align it with cutting-edge industry standards, several features are proposed for future development:

- **Integration of Generative AI:** Future iterations could leverage Agentic AI and Retrieval-Augmented Generation (RAG) architectures to assist in the peer-review process. This includes using AI to analyze manuscript abstracts and automatically recommend the most suitable peer reviewers based on their past publications and expertise.
- **Automated Plagiarism Detection:** Integrating third-party APIs (such as Turnitin) directly into the submission pipeline would allow the system to automatically generate a similarity score during the initial triage phase, saving significant editorial time.
- **Real-time Collaboration Tools:** Implementing WebSockets to allow multiple co-authors to view submission statuses, respond to reviewer comments, and edit metadata in real-time.

REFERENCES

- [1] Prisma Data Platform. "Prisma Postgres" [Online]. Available: <https://www.prisma.io/postgres>
- [2] React Documentation. "React: The Library for Web and Native User Interfaces." [Online]. Available: <https://react.dev/>
- [3] S. Andjelic, S. Obradovic, and B. Gacesa, "A performance analysis of the DBMS - MySQL vs PostgreSQL," *Communications - Scientific Letters of the University of Zilina*, vol. 10, pp. 53-5
- [4] OpenJS Foundation. "Node.js v20.x Documentation." [Online]. Available: <https://nodejs.org/docs/latest-v20.x/api/>
- [5] Tailwind Labs. "Tailwind CSS Documentation." [Online]. Available: <https://tailwindcss.com/docs/>